

WEB 入门—XML&反射

今日内容介绍

- ◆ 使用反射模拟 servlet 执行

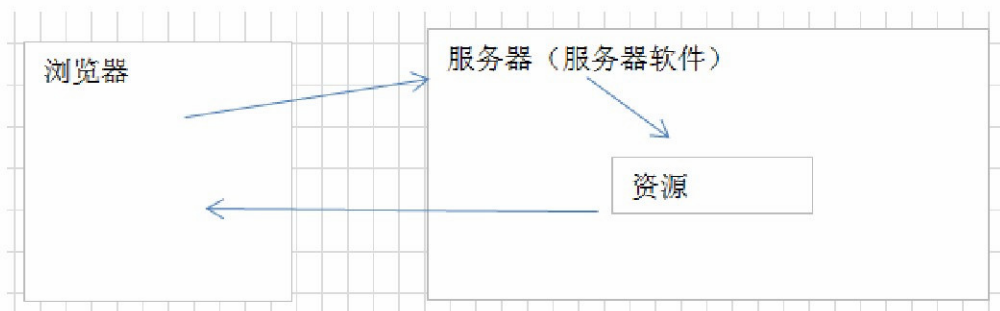
今日内容学习目标

- ◆ 可以编写 xml 存放任意内容
- ◆ 通过 DTD 约束编写指定格式的 XML
- ◆ 通过 Schema 约束编写指定格式的 XML
- ◆ 可以使用 DOM4J 解析 xml
- ◆ 会使用反射对类、方法、构造进行相应操作

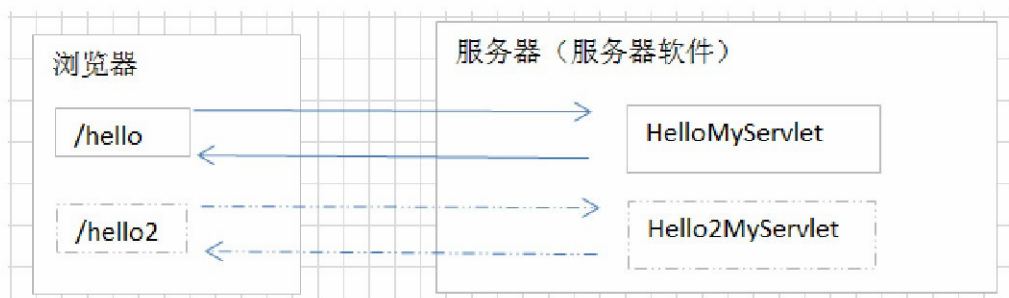
第1章 模拟 servlet 执行

1.1 案例介绍

第三阶段开始，我们将进行 WEB 编程阶段。在 WEB 编程中，我们可以通过浏览器访问 WEB 服务器上的资源。WEB 服务器就是另一台计算机，这样学生就可以访问老师的资源了。



浏览器的入口不同（访问路径），访问的资源也不同。



1.2 相关技术

为了灵活实现的不同路径 (`/hello`) 执行不同的资源 (`HelloMyServlet`) 我们需要使用 XML 进行配置；为了限定 XML 内容，我们需要使用 xml 约束 (DTD 或 schema)；为了获得 xml 的内容，我们需要使用 `dom4j` 进行解析。

1.2.1 XML

1.2.1.1 什么是 XML

XML 全称为 Extensible Markup Language，意思是可扩展的标记语言。XML 语法上和 HTML 比较相似，但 HTML 中的元素是固定的，而 XML 的标签是可以由用户自定义的。

W3C 在 1998 年 2 月发布 1.0 版本，2004 年 2 月又发布 1.1 版本，但因为 1.1 版本不能向下兼容 1.0 版本，所以 1.1 没有人用。同时，在 2004 年 2 月 W3C 又发布了 1.0 版本的第三版。我们要学习的还是 1.0 版本!!!

1.2.1.2 应用常见

- 配置文件

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.5">
    <servlet>
        <servlet-name>HelloMyServlet</servlet-name>
        <servlet-class>cn.itcast.HelloMyServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>HelloMyServlet</servlet-name>
        <url-pattern>/hello</url-pattern>
    </servlet-mapping>
</web-app>
```

- 存放数据

```
<?xml version="1.0" encoding="UTF-8"?>
<persons>
  <person id="p001">
    <name>张三</name>
  </person>
  <person id="p002">
    <name>李四</name>
  </person>
</persons>
```

1.2.1.3 XML 语法

- XML 文档声明

```
<?xml version="1.0" encoding="UTF-8"?>
```

1. 文档声明必须为<?xml 开头，以?>结束；
2. 文档声明必须从文档的 0 行 0 列位置开始；
3. 文档声明只有三个属性：
 - a) version: 指定 XML 文档版本。必须属性，因为我们不会选择 1.1，只会选择 1.0；
 - b) encoding: 指定当前文档的编码。可选属性，默认值是 utf-8；

- 元素 element

```
<servlet>
```

1. 元素是 XML 文档中最重要的组成部分，
2. 普通元素的结构开始标签、元素体、结束标签组成。例如：<hello>大家好</hello>
3. 元素体：元素体可以是元素，也可以是文本，例如：<a>你好
4. 空元素：空元素只有开始标签，而没有结束标签，但元素必须自己闭合，例如：<c/>
5. 元素命名：
 - a) 区分大小写
 - b) 不能使用空格，不能使用冒号：
 - c) 不建议以 XML、xml、Xml 开头
6. 格式化良好的 XML 文档，必须只有一个根元素。

- 属性

```
<web-app version="2.5">
```

1. 属性是元素的一部分，它必须出现在元素的开始标签中
2. 属性的定义格式：属性名=属性值，其中属性值必须使用单引或双引
3. 一个元素可以有 0~N 个属性，但一个元素中不能出现同名属性
4. 属性名不能使用空格、冒号等特殊字符，且必须以字母开头

- 注释

XML 的注释与 HTML 相同，即以“<!--”开始，以“-->”结束。注释内容会被 XML 解析器忽略！

- 转义字符

XML 中的转义字符与 HTML 一样。

因为很多符号已经被 XML 文档结构所使用，所以在元素体或属性值中想使用这些符号就必须使用转义字符，例如：“<”、“>”、“'”、“””、“&”。

字符	字符引用 (十进制代码)	字符引用 (十六进制代码)	预定义实体引用
<	<	<	<
>	>	>	>
"	"	"	"
'	'	'	'
&	&	&	&

- CDATA 区

```
<![CDATA[  
    任意内容  
]]>
```

当大量的转义字符出现在 xml 文档中时，会使 xml 文档的可读性大幅度降低。这时如果使用 CDATA 段就会好一些。

在 CDATA 段中出现的“<”、“>”、“””、“'”、“&”，都无需使用转义字符。这可以提高 xml 文档的可读性。

在 CDATA 段中不能包含“]]>”，即 CDATA 段的结束定界符。

1.2.2 DTD 约束

- 常见的 xml 约束：DTD、Schema

1.2.2.1 什么是 DTD

DTD（Document Type Definition），文档类型定义，用来约束 XML 文档。规定 XML 文档中元素的名称，子元素的名称及顺序，元素的属性等。

1.2.2.2 DTD 重点要求

开发中，我们很少自己编写 DTD 约束文档，通常情况我们都是通过框架提供的 DTD 约束文档，

编写对应的 XML 文档。常见框架使用 DTD 约束有：struts2、hibernate 等。

通过提供的 DTD “web-app_2_3.dtd” 编写 XML

```
<?xml version="1.0" encoding="UTF-8"?>
<!--
    传智播客 DTD 教学实例文档。
    模拟 servlet2.3 规范，如果开发人员需要在 xml 使用当前 DTD 约束，必须包括 DOCTYPE。
    格式如下：
    <!DOCTYPE web-app SYSTEM "web-app_2_3.dtd">
-->
<!ELEMENT web-app (servlet*,servlet-mapping* , welcome-file-list?) >
<!ELEMENT servlet (servlet-name,description?,(servlet-class|jsp-file))>
<!ELEMENT servlet-mapping (servlet-name,url-pattern) >
<!ELEMENT servlet-name (#PCDATA)>
<!ELEMENT servlet-class (#PCDATA)>
<!ELEMENT url-pattern (#PCDATA)>
<!--
    欢迎文件列表
-->
<!ELEMENT welcome-file-list (welcome-file+)>
<!ELEMENT welcome-file (#PCDATA)>
<!--
    版本
-->
<!ATTLIST web-app version CDATA #IMPLIED>
```

1.2.2.3 案例实现

- 步骤 1: 创建 web.xml 文档, 并将 “web-app 2 3.dtd” 拷贝相同目录下。

- cn.itcast.b_dtd
 - web-app_2_3.dtd
 - web.xml

- 步骤 2: 从 DTD 文档开始处, 拷贝需要的“文档声明”

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!--
3     传智播客DTD教学实例文档。
4     模拟servlet2.3规范，如果开发人员需要在xml使用当前DTD约束，必须包括DOCTYPE。
5     格式如下：
6     <!DOCTYPE web-app SYSTEM "web-app_2_3.dtd">
7 -->
```



web-app_2_3.dtd web.xml

```
1 <?xml version='1.0' encoding='UTF-8'?>
2 <!DOCTYPE web-app SYSTEM 'web-app_2_3.dtd'>
```

- ### ● 步骤 3: 完成 xml 内容编写

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE web-app SYSTEM "web-app_2_3.dtd">
<web-app version="2.3">
```

```
<servlet>
    <servlet-name></servlet-name>
    <servlet-class></servlet-class>
</servlet>
<servlet-mapping>
    <servlet-name></servlet-name>
    <url-pattern></url-pattern>
</servlet-mapping>

<welcome-file-list>
    <welcome-file></welcome-file>
    <welcome-file></welcome-file>
    <welcome-file></welcome-file>
</welcome-file-list>
</web-app>
```

1.2.2.4 DTD 语法（了解）

1.2.2.4.1 文档声明

1. 内部 DTD，在 XML 文档内部嵌入 DTD，只对当前 XML 有效。

```
<?xml version="1.0" encoding="utf-8" standalone="yes" ?>
<!DOCTYPE web-app [
    ... //具体的语法
]>
<web-app>
</web-app>
```

2. 外部 DTD—本地 DTD，DTD 文档在本地系统上，公司内部自己项目使用。

```
<?xml version="1.0" encoding="utf-8" standalone="no" ?>
<!DOCTYPE web-app SYSTEM "web-app_2_3.dtd">
<web-app>
</web-app>
```

3. 外部 DTD—公共 DTD，DTD 文档在网络上，一般都有框架提供。

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE web-app PUBLIC "-//Sun Microsystems, Inc.//DTD Web Application 2.3//EN"
"http://java.sun.com/dtd/web-app_2_3.dtd">
<web-app>
</web-app>
```

1.2.2.4.2 元素声明

定义元素语法: `<!ELEMENT 元素名 元素描述>`

元素名: 自定义

元素描述包括: 符号和数据类型

常见符号: `? * + () | ,`

常见类型: `#PCDATA` 表示内容是文本, 不能是子标签

符号	符号类型	描述	示例
?	问号	表示该对象可以出现, 但只能出现一次	(菜鸟?)
*	星号	表示该对象允许出现任意多次, 也可以是零次	(爱好*)
+	加号	表示该对象最少出现一次, 可以出现多次	(成员+)
()	括号	用来给元素分组	(古龙 金庸 梁羽生), (王朔 余杰), 毛毛
	竖条	表明在列出的对象中选择一个	(男人 女人)
,	逗号	表示对象必须按指定的顺序出现	(西瓜, 苹果, 香蕉)

● 实例

```
<!ELEMENT web-app (servlet*,servlet-mapping* , welcome-file-list?) >
    web-app 包括 3 个标签, 且必须顺序出现。
        servlet 子标签个数任意
        servlet-mapping 子标签个数任意
        welcome-file-list 子标签最多只能出现一次

<!ELEMENT servlet (servlet-name,description?,(servlet-class|jsp-file))>
    servlet 有 3 个子标签, 且必须顺序出现
        servlet-name, 必须有, 且只能出现一次
        description, 可选一次
        servlet-class 和 jsp-file 二选一, 且只能出现一次

<!ELEMENT servlet-name (#PCDATA)>
    servlet-name 的标签体必须是文本

<!ELEMENT welcome-file-list (welcome-file+)>
    welcome-file-list 至少有一个子标签 welcome-file
```

1.2.2.4.3 属性声明

属性的语法: (attribute)

`<!ATTLIST 元素名`

属性名 属性类型 约束

属性名 属性类型 约束

...

```
>
元素名：属性必须是给元素添加，所有必须先确定元素名
属性名：自定义
属性类型：ID、CDATA、枚举 ...
    ID：ID 类型的属性用来标识元素的唯一性
    CDATA：文本类型
    枚举：(e1 | e2 | ...) 多选一
约束：
    #REQUIRED：说明属性是必须的；required
    #IMPLIED：说明属性是可选的；implied
```

● 实例

```
<!ATTLIST web-app version CDATA #IMPLIED>
给 web-app 元素添加 version 属性，属性值必须是文本，且可选。
<web-app version="2.3"> 和 <web-app> 都符号约束
```

1.2.3 Schema 约束

1.2.3.1 什么是 Schema

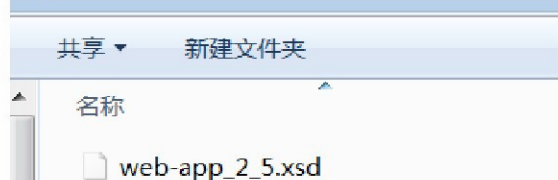
Schema 是新的 XML 文档约束；
Schema 要比 DTD 强大很多，是 DTD 替代者；
Schema 本身也是 XML 文档，但 Schema 文档的扩展名为 xsd，而不是 xml。
Schema 功能更强大，数据类型更完善
Schema 支持名称空间

1.2.3.2 Schema 重点要求

与 DTD 一样，要求可以通过 schema 约束文档编写 xml 文档。常见框架使用 schema 的有：Spring 等

通过提供“web-app_2_5.xsd”编写 xml 文档

day08--xml&tomcat\资料\02.Schema约束文档



```
<?xml version="1.0" encoding="UTF-8"?>
<!--
```

传智播客 Schema 教学实例文档。

模拟 servlet2.5 规范，如果开发人员需要在 xml 使用当前 Schema 约束，必须包括指定命名空间。

格式如下:

```
<web-app xmlns="http://www.example.org/web-app_2_5"
          xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
          xsi:schemaLocation="http://www.example.org/web-app_2_5
web-app_2_5.xsd"
          version="2.5">

-->
<xsd:schema xmlns="http://www.w3.org/2001/XMLSchema"
            targetNamespace="http://www.example.org/web-app_2_5"
            xmlns:xsd="http://www.w3.org/2001/XMLSchema"
            xmlns:tns="http://www.example.org/web-app_2_5"
            elementFormDefault="qualified">

  <xsd:element name="web-app">
    <xsd:complexType>
      <xsd:choice minOccurs="0" maxOccurs="unbounded">
        <xsd:element name="servlet">
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element name="servlet-name"></xsd:element>
              <xsd:element name="servlet-class"></xsd:element>
            </xsd:sequence>
          </xsd:complexType>
        </xsd:element>
        <xsd:element name="servlet-mapping">
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element name="servlet-name"></xsd:element>
              <xsd:element name="url-pattern"></xsd:element>
            </xsd:sequence>
          </xsd:complexType>
        </xsd:element>
        <xsd:element name="welcome-file-list">
          <xsd:complexType>
            <xsd:sequence>
              <xsd:element
name="welcome-file"
maxOccurs="unbounded"></xsd:element>
            </xsd:sequence>
          </xsd:complexType>
        </xsd:element>
      </xsd:choice>
      <xsd:attribute
name="version"
type="double"
use="optional"></xsd:attribute>
    </xsd:complexType>
```

```
</xsd:element>
</xsd:schema>
```

- 案例文档中同一个“命名空间”分别使用“默认命名空间”和“显示命名空间”进行引入，所以文档中<schema>和<xsd:schema>作用一样。

```
11 <xsd:schema xmlns="http://www.w3.org/2001/XMLSchema" 默认 <schema>
12     targetNamespace="http://www.example.org/web-app_2_5"
13     xmlns:xsd="http://www.w3.org/2001/XMLSchema" 显示 <xsd:schema>
14     xmlns:tns="http://www.example.org/web-app_2_5" 两种都可以
15     elementFormDefault="qualified">
16
```

1.2.3.3 案例实现

1. 步骤 1: 创建 web.xml，并将“web-app_2_5.xsd”拷贝到同级目录

```
cn.itcast.c_schema
├── web-app_2_5.xsd
└── web.xml
```

2. 步骤 2: 从 xsd 文档中拷贝需要的“命名空间”

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <!--
3     传智播客Schema教学实例文档。
4     模拟servlet2.5规范，如果开发人员需要在xml使用当前Schema约束，必须包括指定命名空间。
5     格式如下：
6     <web-app xmlns="http://www.example.org/web-app_2_5"
7             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
8             xsi:schemaLocation="http://www.example.org/web-app_2_5 web-app_2_5.xsd"
9             version="2.5">
10 -->
```

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <web-app xmlns="http://www.example.org/web-app_2_5"
3         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4         xsi:schemaLocation="http://www.example.org/web-app_2_5 web-app_2_5.xsd"
5         version="2.5">
6
7     <servlet>
```

3. 完成 xml 内容编写

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app xmlns="http://www.example.org/web-app_2_5"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://www.example.org/web-app_2_5 web-app_2_5.xsd"
        version="2.5">

    <servlet>
        <servlet-name></servlet-name>
```

```
<servlet-class></servlet-class>
</servlet>

<servlet-mapping>
  <servlet-name></servlet-name>
  <url-pattern></url-pattern>
</servlet-mapping>

<welcome-file-list>
  <welcome-file></welcome-file>
  <welcome-file></welcome-file>
  <welcome-file></welcome-file>
</welcome-file-list>

</web-app>
```

1.2.3.4 命名空间（语法）

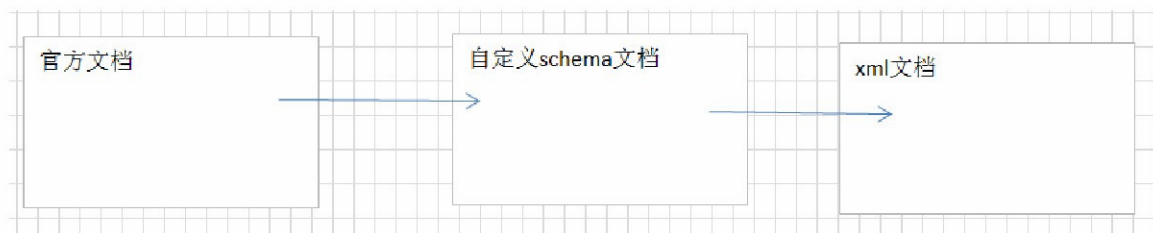
1.2.3.4.1 什么是命名空间

如果一个 XML 文档中使用多个 Schema 文件，而这些 Schema 文件中定义了相同名称的元素时就会出现名字冲突。这就像一个 Java 文件中使用了 `import java.util.*` 和 `import java.sql.*` 时，在使用 `Date` 类时，那么就不明确 `Date` 是哪个包下的 `Date` 了。

总之名称空间就是用来处理元素和属性的名称冲突问题，与 Java 中的包是同一用途。如果每个元素和属性都有自己的名称空间，那么就不会出现名字冲突问题，就像是每个类都有自己所在的包一样，那么类名就不会出现冲突。

1.2.3.4.2 约束文档和 XML 关系

当 W3C 提出 Schema 约束规范时，就提供“官方约束文档”。我们通过官方文档，必须“自定义 schema 约束文档”，开发中“自定义文档”由框架编写者提供。我们提供“自定义文档”限定，编写出自己的 xml 文档。



1.2.3.4.3 声明命名空间

默认命名空间: `<xxx xmlns="">` , 使用`<标签>`

显式命名空间: `<xxx xmlns:别名="">` , 使用`<别名:标签>`

- 实例: web-app_2_5.xsd

```
<xsd:schema xmlns:xsd="http://www.w3.org/2001/XMLSchema" ...>
```

表示自定义 schema 约束文档引用官方文档作为显示命名空间。如果要使用官方提供的元素或属性, 必须使用 xsd 前缀 (自定义, 此处表示官方文档, 所以使用 xsd)

`<xsd:schema>` 标签就有官方文档提供, 默认命名空间直接使用。

- 实例: web.xml

```
<web-app xmlns=http://www.example.org/web-app_2_5 ...>
```

表示 xml 文档引用 “自定义约束文档” 作为默认命名空间

因为使用默认命名空间, `<web-app>` 直接使用

1.2.3.4.4 其他介绍

- 自定义约束: web-app_2_5.xsd

```
<xsd:schema targetNamespace=http://www.example.org/web-app_2_5
```

表示给当前自定义约束文档进行起名, 提供给 xml 文档使用。

- xml 文档: web.xml

```
<web-app
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation=http://www.example.org/web-app_2_5 web-app_2_5.xsd
```

`xmlns:xsi=""` 固定写法

表示是一个 schema 实例文档, 就是被 schema 文档约束的 xml 文档。

`xsi:schemaLocation=""` 名称 路径 名称 路径 名称 路径 ...

表示用于确定当前 xml 文档使用到的 schema 文档的位置。“名称 路径”是成对出现, 与 xmlns 引用命名空间对应。

- 综合案例: (了解)

\\day11--xml\\资料\\03.Schema综合案例

共享 新建文件夹

名称



spring doc

约束头信息参考文档



applicationContext.xml

案例

综合案例目的是: 一个 “命名空间” 必须在 schemaLocation 中有一个成对描述信息 (名称 路径)

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xmlns:context="http://www.springframework.org/schema/context"
5       xmlns:aop="http://www.springframework.org/schema/aop"
6       xmlns:tx="http://www.springframework.org/schema/tx"
7       xsi:schemaLocation="http://www.springframework.org/schema/beans
8                           http://www.springframework.org/schema/beans/spring-beans.xsd
9                           http://www.springframework.org/schema/context
10                          http://www.springframework.org/schema/context/spring-context.xsd
11                          http://www.springframework.org/schema/tx
12                          http://www.springframework.org/schema/tx/spring-tx.xsd
13                          http://www.springframework.org/schema/aop
14                          http://www.springframework.org/schema/aop/spring-aop.xsd">
15
```

```
2 <beans xmlns="http://www.springframework.org/schema/beans"
3       xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4       xmlns:context="http://www.springframework.org/schema/context"
5       xmlns:aop="http://www.springframework.org/schema/aop"
6       xmlns:tx="http://www.springframework.org/schema/tx"
7       xsi:schemaLocation="http://www.springframework.org/schema/beans
8                           http://www.springframework.org/schema/beans/spring-beans.xsd
9                           http://www.springframework.org/schema/context
10                          http://www.springframework.org/schema/context/spring-context.xsd
11                          http://www.springframework.org/schema/tx
12                          http://www.springframework.org/schema/tx/spring-tx.xsd
13                          http://www.springframework.org/schema/aop
14                          http://www.springframework.org/schema/aop/spring-aop.xsd">
```

1.2.4 dom4j 解析

1.2.4.1 XML 解析概述

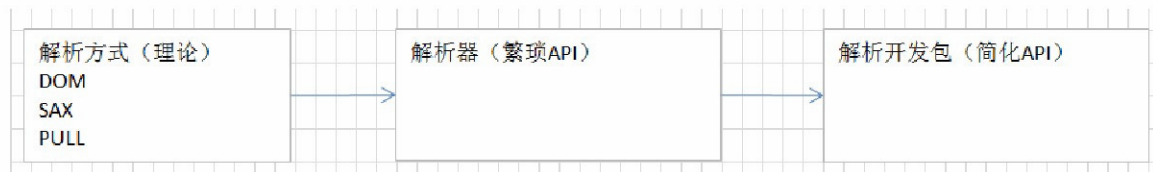
当将数据存储在 XML 后，我们就希望通过程序获得 XML 的内容。如果我们使用 Java 基础所学习的 IO 知识是可以完成的，不过你需要非常繁琐的操作才可以完成，且开发中会遇到不同问题（只读、读写）。人们为不同问题提供不同的解析方式，并提交对应的解析器，方便开发人员操作 XML。

1.2.4.2 解析方式和解析器

● 开发中比较常见的解析方式有三种，如下：

1. DOM：要求解析器把整个 XML 文档装载到内存，并解析成一个 Document 对象。
 - a) 优点：元素与元素之间保留结构关系，故可以进行增删改查操作。
 - b) 缺点：XML 文档过大，可能出现内存溢出显现。
2. SAX：是一种速度更快，更有效的方法。它逐行扫描文档，一边扫描一边解析。并以事件驱动的方式进行具体解析，每执行一行，都将触发对应的事件。（了解）
 - a) 优点：处理速度快，可以处理大文件
 - b) 缺点：只能读，逐行后将释放资源。
3. PULL：Android 内置的 XML 解析方式，类似 SAX。（了解）

- 解析器：就是根据不同的解析方式提供的具体实现。有的解析器操作过于繁琐，为了方便开发人员，有提供易于操作的解析开发包。



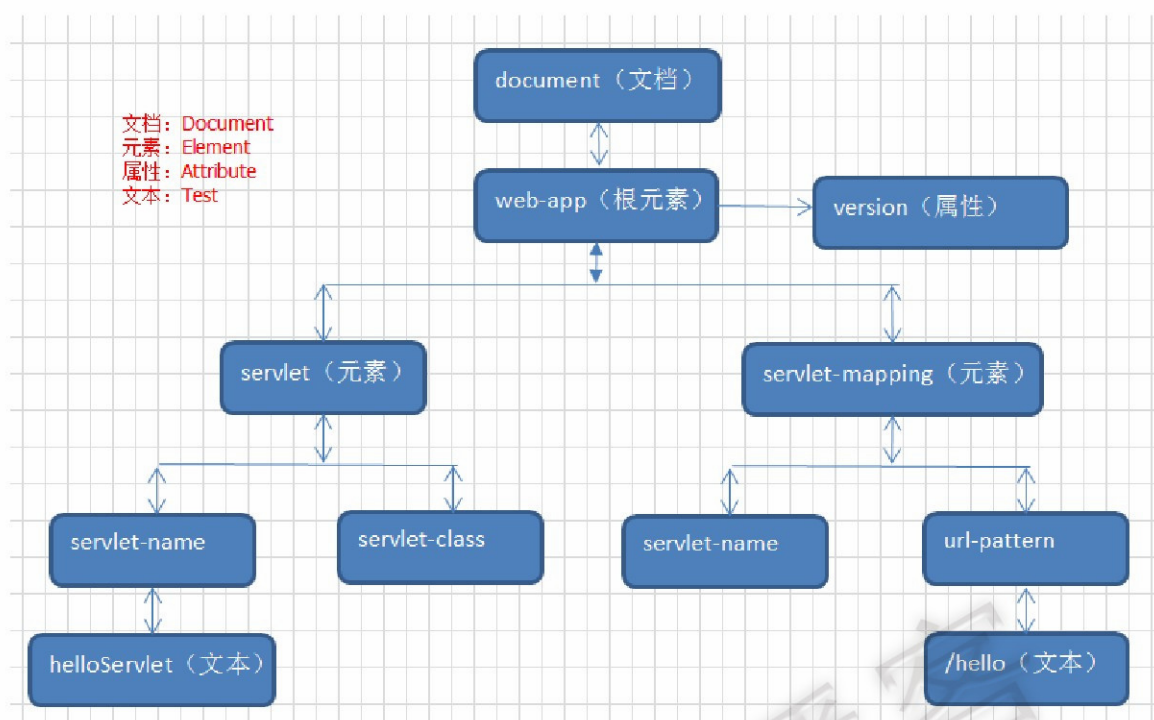
- 常见的解析开发包：
 - JAXP: sun 公司提供支持 DOM 和 SAX 开发包
 - JDom: dom4j 兄弟
 - jsoup: 一种处理 HTML 特定解析开发包
 - dom4j: 比较常用的解析开发包，hibernate 底层采用。

1.2.4.3 DOM 解析原理及结构模型

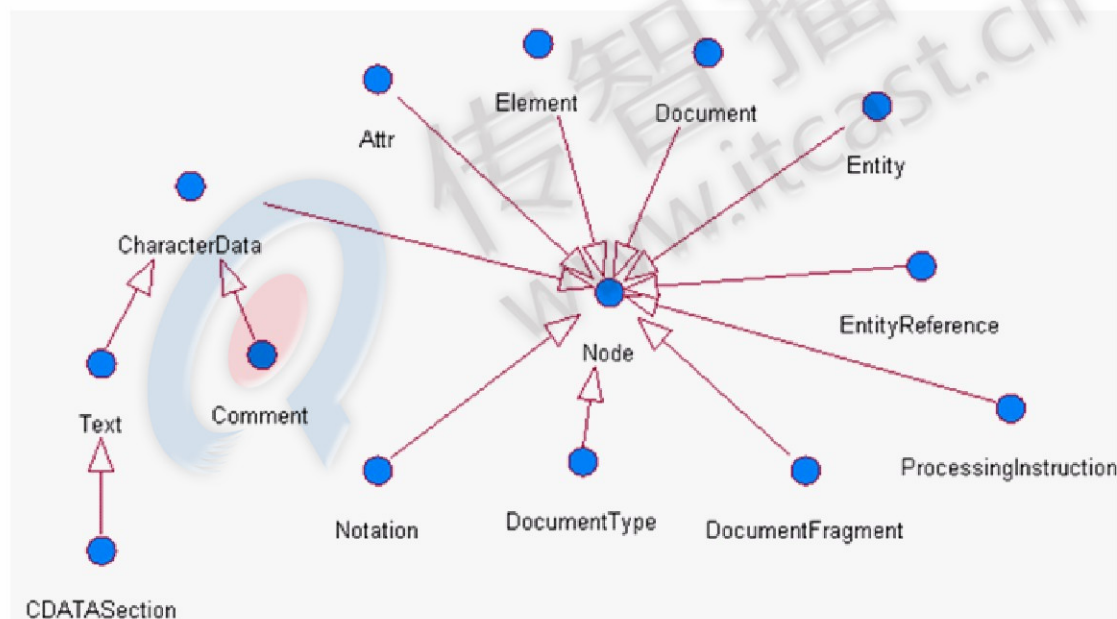
XML DOM 和 HTML DOM 类似，XML DOM 将 整个 XML 文档加载到内存，生成一个 DOM 树，并获得一个 Document 对象，通过 Document 对象就可以对 DOM 进行操作

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.5">
  <servlet>
    <servlet-name>helloServlet</servlet-name>
    <servlet-class>cn.itcast.HelloServlet</servlet-class>
  </servlet>
  <servlet-mapping>
    <servlet-name>helloServlet</servlet-name>
    <url-pattern>/hello</url-pattern>
  </servlet-mapping>
</web-app>
```

The screenshot shows a code editor with a file named 'web.xml'. The code is an XML document defining a web application. It starts with an XML declaration, followed by a root element 'web-app' containing a 'servlet' element (with 'servlet-name' and 'servlet-class' attributes) and a 'servlet-mapping' element (with 'servlet-name' and 'url-pattern' attributes). The code is highlighted with a blue background.



DOM 中的核心概念就是节点，在 XML 文档中的元素、属性、文本等，在 DOM 中都是节点！



1.2.4.4 API 使用

如果需要使用 dom4j，必须导入 jar 包。

- Referenced Libraries
 - dom4j-1.6.1.jar
- lib
 - dom4j-1.6.1.jar

dom4j 必须使用核心类 SaxReader 加载 xml 文档获得 Document，通过 Document 对象获得文档

的根元素，然后就可以操作了。

常用 API 如下：

1. SaxReader 对象
 - a) read(...) 加载执行 xml 文档
2. Document 对象
 - a) getRootElement() 获得根元素
3. Element 对象
 - a) elements(...) 获得指定名称的所有子元素。可以不指定名称
 - b) element(...) 获得指定名称第一个子元素。可以不指定名称
 - c) getName() 获得当前元素的元素名
 - d) attributeValue(...) 获得指定属性名的属性值
 - e) elementText(...) 获得指定名称子元素的文本值
 - f) getText() 获得当前元素的文本内容

```
@Test
public void demo03() throws Exception{
    // #1 获得 document
    SAXReader saxReader = new SAXReader();
    Document document = saxReader.read(new File("src/cn/itcast/a_xml/web.xml"));

    // #2 获得根元素
    Element rootElement = document.getRootElement();

    // 打印 version 属性值
    String version = rootElement.attributeValue("version");
    System.out.println(version);

    // #3 获得所有子元素。例如：<servlet>/<servlet-mapping>
    List<Element> allChildElement = rootElement.elements();

    // #4 遍历所有
    for (Element childElement : allChildElement) {
        // #5.1 打印元素名
        String childEleName = childElement.getName();
        System.out.println(childEleName);

        // #5.2 处理<servlet> , 并获得子标签的内容。例如：<servlet-name> 等
        if ("servlet".equals(childEleName)){
            // 方式 1：获得元素对象，然后获得文本
            Element servletNameElement = childElement.element("servlet-name");
            String servletName = servletNameElement.getText();
            System.out.println("\t" + servletName);

            // 方式 2：获得元素文本值
```

```
        String servletClass = childElement.elementText("servlet-class");
        System.out.println("\t" + servletClass);

    }

    // #5.3 处理<servlet-mapping> 省略...

}

}
```

1.2.5 反射

- 什么是反射
 - JAVA 反射机制是在运行状态中，对于任意一个类，都能够知道这个类的所有属性和方法；对于任意一个对象，都能够调用它的任意一个方法和属性
 - 使用反射，可以在运行时对类 **Class**、构造方法 **Constructor**、普通方法 **Method**、字段 **Field** 进行操作。
- **Class 对象**
 - **Class 对象**，是对 **class 文件**（字节码文件）的描述对象。
 - 获得 **Class 对象**
 - ◆ 已经类的完整路径名(字符串)：**Class.forName(..)**
 - ◆ 已经类型：**String.class**，确定参数列表
 - ◆ 已经对象：**obj.getClass()**
 - 常用方法
 - ◆ 使用默认构造方法创建实例：**newInstance()**
- **Constructor 对象**
 - **Constructor 对象**，是构造方法的描述对象。
 - 获得构造方法
 - ◆ 公共的构造方法：**Constructor<T> getConstructor(Class<?>... parameterTypes)**，可变参数用于确定形式参数列表
 - ◆ 已经声明的构造方法：**Constructor<T> getDeclaredConstructor(Class<?>... parameterTypes)**，获得私有的构造
 - 实例化对象实例
 - ◆ **newInstance(Object... initargs)**，可变参数用于确定实际参数列表
- **Method 对象**
 - **Method 对象**，是普通方法的描述对象
 - 获得方法：
 - ◆ 获得公共方法：**Method getMethod(String name, Class<?>... parameterTypes)** 通过方法 **name** 获得方法，可变参数为方法的形式参数列表

- ◆ 获得声明方法: `Method getDeclaredMethod(String name, Class<?>... parameterTypes)` 方法操作:
- 执行指定方法
 - ◆ `Object invoke(Object obj, Object... args)` 执行指定对象 `obj`, 指定方法, 可变参数为方法的实际参数列表
- Field 对象
 - Field 对象, 是字段的描述对象
 - 获得方法
 - ◆ 所有字段: `Field getField(String name)` , 通过字段名称
 - ◆ 声明字段: `Field getDeclaredField(String name)`
 - 操作
 - ◆ 获得内容: `Object get(Object obj)`
 - ◆ 设置内容: `void set(Object obj, Object value)` , 确定实例对象

1.3 案例实现

1.3.1 准备工作

为了模拟服务器端程序, 且可以同时存在多个类似程序。故提供接口, 接口中有 3 个方法, 我们人为约定三个方法的调用顺序。

```
public interface MyServlet {  
  
    public void init();    //1.初始化  
  
    public void service(); //2.执行  
  
    public void destory(); //3.销毁  
  
}
```

在为接口编写实现类

```
└─ cn.itcast.e_web  
    └─ HelloMyServlet.java  
        └─ MyServlet.java
```

```
public class HelloMyServlet implements MyServlet {  
  
    @Override  
    public void init() {  
        System.out.println("1.初始化");  
    }  
}
```

```
@Override
public void service() {
    System.out.println("2.执行中....");
}

@Override
public void destory() {
    System.out.println("3.销毁");
}

}
```

测试，创建实现类实例对象

```
public class TestApp {
    @Test
    public void demo01(){
        //手动创建执行
        MyServlet myServlet = new HelloMyServlet();
        myServlet.init();
        myServlet.service();
        myServlet.destory();
    }
}
```

1.3.2 反射执行

测试程序我们直接 new HelloServlet，这种编程方式我们称为硬编码，及代码写死了。为了后期程序的可扩展，开发中通常使用实现类的全限定类名（cn.itcast.e_web.HelloMyServlet），通过反射加载字符串指定的类，并通过反射创建实例。

```
@Test
public void demo02() throws Exception{
    /* 反射创建执行
    * 1) Class.forName() 返回指定接口或类的 Class 对象
    * 2) newInstance() 通过 Class 对象创建类的实例对象，相当于 new Xxx();
    */
    String servletClass = "cn.itcast.e_web.HelloMyServlet";

    //3 获得字符串实现类实例
    Class clazz = Class.forName(servletClass);
    MyServlet myServlet = (MyServlet) clazz.newInstance();
    //4 执行对象的方法
    myServlet.init();
    myServlet.service();
    myServlet.destory();
}
```

```
}
```

1.3.3 解析 xml

使用反射我们已经可以创建对象的实例，此时我们使用的全限定类名，在程序是仍写死了，我们将器配置到 xml 文档中。

xml 文档内容：

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="2.5">
    <servlet>
        <servlet-name>HelloMyServlet</servlet-name>
        <servlet-class>cn.itcast.e_web.HelloMyServlet</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>HelloMyServlet</servlet-name>
        <url-pattern>/hello</url-pattern>
    </servlet-mapping>

    <servlet>
        <servlet-name>HelloMyServlet2</servlet-name>
        <servlet-class>cn.itcast.e_web.HelloMyServlet2</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>HelloMyServlet2</servlet-name>
        <url-pattern>/hello2</url-pattern>
    </servlet-mapping>
</web-app>
```

解析实现

```
@Test
public void demo03() throws Exception{
    /* 读取 xml 配置文件，获得<servlet-class>配置的内容，取代固定字符串
    */

    //1.1 加载 xml 配置文件，并获得 document 对象
    SAXReader saxReader = new SAXReader();
    Document document = saxReader.read(new File("src/cn/itcast/e_web/web.xml"));
    //1.2 获得根元素
    Element rootElement = document.getRootElement();
    //1.3 获得第一个<servlet> 子元素
    Element servletElement = rootElement.element("servlet");
    //1.4 获得字符串实现类 <servlet-class>的值
    String servletClass = servletElement.elementText("servlet-class");
```

```

//3 获得字符串实现类实例
Class clazz = Class.forName(servletClass);
MyServlet myServlet = (MyServlet) clazz.newInstance();
//4 执行对象的方法
myServlet.init();          //或 clazz.getMethod("init").invoke(myServlet);
myServlet.service();
myServlet.destory();
}

```

1.3.4 模拟浏览器路径

上面我们已经解析 xml，不过我们获得内容是固定。我们希望如果用户访问的路径是/hello，将执行 cn.itcast.e_web.HelloMyServlet 程序，如果访问时/hello2，将执行 cn.itcast.e_web.HelloMyServlet2 程序。

在执行测试程序前（@Before），解析 xml 文件，将解析的结果存放在 Map 中，map 中数据的格式为：路径=实现类。

```

<servlet>
  <servlet-name>HelloMyServlet</servlet-name>
  <servlet-class>cn.itcast.e_web.HelloMyServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>HelloMyServlet</servlet-name>
  <url-pattern>/hello</url-pattern>
</servlet-mapping>

```

map 的 value

map 的 key

解析 xml 思路：先解析<servlet>，将结果存放 map，name=class，然后再解析<servlet-mapping>通过 name 获得 class，再将 url=class 存放到 map，最后将 name=class 移除。

```

//最终存放 key=请求路径，value=实现类
private Map<String, String> data = new HashMap<String,String>();
@Before
public void demo04Before() throws Exception{
    //在执行前执行，解析 xml，并将结果存放到 Map<路径，实现类>中
    //1 获得 document
    SAXReader saxReader = new SAXReader();
    Document document = saxReader.read(new File("src/cn/itcast/e_web/web.xml"));
    //2 获得根元素
    Element rootElement = document.getRootElement();
    //3 获得所有的子元素 <servlet>、<servlet-mapping>等
    List<Element> allChildElement = rootElement.elements();
    /* 4 遍历所有
     * 1)解析到<servlet>，将其子标签<servlet-name>与<servlet-class>存放到 Map 中
     * 2)解析到<servlet-mapping>，获得子标签<servlet-name>和<url-pattern>，从 map 中获得 1 的内容，组合成 url = class 键值对
     */
    for (Element childElement : allChildElement) {

```

```
//4.1 获得元素名
String eleName = childElement.getName();

//4.2 如果是 servlet，将解析内容存放到 Map 中
if("servlet".equals(eleName)){
    String servletName = childElement.elementText("servlet-name");
    String servletClass = childElement.elementText("servlet-class");
    data.put(servletName, servletClass);
}

//4.3 如果是 servlet-mapping，获得之前内容，组成 key=url,value=class 并添加到
Map 中
if("servlet-mapping".equals(eleName)){
    String servletName = childElement.elementText("servlet-name");
    String urlPattern = childElement.elementText("url-pattern");
    // 获得<servlet-name>之前存放在 Map 中<servlet-class>值
    String servletClass= data.get(servletName);
    // 存放新的内容 url = class
    data.put(urlPattern, servletClass);
    // 将之前存放的数据删除
    data.remove(servletName);
}

//打印信息
System.out.println(data);
}
}
```

模拟浏览器请求路径，通过 url 从 map 获得 class，并使用反射执行实现类。

```
@Test
public void demo04() throws Exception{

    //1 模拟路径
    String url = "/hello";
    //    String url = "/hello2";

    //2 通过路径获得对应的实现类
    String servletClass = data.get(url);

    //3 获得字符串实现类实例
    Class clazz = Class.forName(servletClass);
    MyServlet myServlet = (MyServlet) clazz.newInstance();
    //4 执行对象的方法
    myServlet.init();
    myServlet.service();
    myServlet.destory();
}
```

}

第2章 总结

